Postfork: Bitcoin Core with CTV and CSFS from Genesis, Backed Only by Pegged Bitcoin

Postfork

postfork.org (planned)

Design draft · specification edition spec-v1.0 (draft) · 2026-11-08

Status. Pre-launch design specification. No code. Not production-ready. Nothing described here is live, and nothing may be called pBTC (Postfork) until every launch gate (Section 11) has cleared. All numbers are provisional launch values. Neither CTV nor CSFS is active on Bitcoin. This paper is informative only; the single normative document is the Postfork specification `spec/POSTFORK.md` [24]. Where this paper simplifies or differs, the specification wins.

Abstract. Bitcoin script with the proposed covenant opcodes CTV and CSFS would let anyone build vaults, payment trees, channels and delegation directly in script, but today those opcodes run only on test networks with valueless coins. We propose Postfork, a new blockchain from block 0 whose consensus is Bitcoin Core with BIP-119 (`OP_CHECKTEMPLATEVERIFY`) and BIP-348 (`OP_CHECKSIGFROMSTACK`) active from genesis, plus a small, explicitly delimited peg extension. Postfork has no block subsidy, no premine and no native token. Its only asset is pBTC (Postfork): bitcoin pegged in from Bitcoin. The peg follows the zkBTC design. The reserve sits in Taproot vaults with a NUMS internal key, spendable only along a pre-signed BitVM2 graph. Anyone may register to join the operator set; the current members certify each epoch and can censor admission. The set only grows, and every new vault is signed by all current members. Postfork consensus tracks the Bitcoin header chain and mints pBTC only against a vault deposit buried at least 2016 Bitcoin blocks deep under a fixed minimum of Bitcoin work. A holder redeems by burning first; an operator of a matching vault's epoch set can pay out and is reimbursed through a claim that anyone can challenge, but no payout is guaranteed. Under stated assumptions, which include conditions on block production and exclude Bitcoin reorganizations that orphan a settled deposit and Postfork reorganizations that orphan an already reimbursed burn, operators cannot steal the reserve, and the worst case is a freeze, which is not theft but may never be recoverable; outside them, unbacked pBTC can drain the reserve. How Postfork blocks are produced is an open decision, and the peg can be no stronger than what its verifier can establish about Postfork.

1. Introduction

Bitcoin Script can check signatures and timelocks, but it cannot inspect outputs directly and cannot enforce a persistent, keyless output covenant without an ongoing signer. Today that typically requires pre-signed transactions plus key deletion, as the BitVM2 graph of this design shows; covenant opcodes such as CTV let an output enforce a transaction template directly. One

concrete proposal, published in 2025 under the name "CTV + CSFS" [11], consists of exactly two BIPs: BIP-119 `OP_CHECKTEMPLATEVERIFY` [2] and BIP-348 `OP_CHECKSIGFROMSTACK` [3]. Both are `Draft`. Reference implementations exist for regression testing and on the Bitcoin Inquisition signet [12]; none is active on Bitcoin. Anyone who wants to build with these opcodes today builds with coins that are worth nothing.

What is needed is a place where the script layer is exactly the proposed one, where the only money is bitcoin, and where no operator other than oneself has to be trusted with that bitcoin, under assumptions stated here, including assumptions about block production. This paper proposes such a place. Postfork is a separate chain with its own genesis and an empty UTXO set. Its rules are the rules of the pinned Bitcoin Core release, with BIP-119 and BIP-348 active from block 0. It has no block subsidy, no premine and no token to buy. Its only asset is pBTC (Postfork), a satoshi-denominated claim on BTC held in a BitVM2 reserve on Bitcoin. The design follows one root principle:

> *Bitcoin as it will look if CTV and CSFS activate — designed to be usable with pegged bitcoin once all launch gates clear, with no new asset, and with a peg in which the only operator you need to trust can be you.*

The peg design is taken from zkBTC [9], a BitVM2-backed bitcoin token on the zkCoins protocol [10], and is cited, not rebranded. The zkCoins family requires no separate chain; Postfork *is* a separate chain, and it stays consistent with that family only by keeping what matters: bitcoin as the only asset, Bitcoin Core rules except for listed deltas, and zkBTC's trust model for the peg.

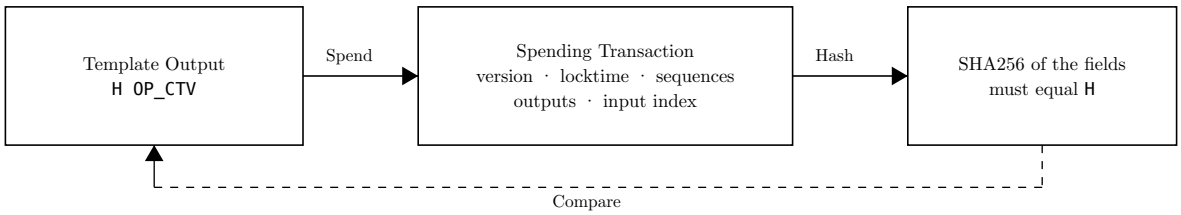
Postfork is not Bitcoin, and pBTC is not BTC. Postfork is not a prediction of what Bitcoin will activate, it is not private, and it is not a rollup. An unrelated token with the ticker pBTC exists: the pTokens wrapped bitcoin of the pNetwork project, whose BSC variant was exploited in 2021 [21]. Postfork has no relation to it.

No deployed mechanism gives a fungible, multi-holder, BTC-backed token a third-party-free unilateral exit to Bitcoin. Federated pegs require a signer quorum to release BTC: Liquid relies on two thirds of its watchmen and has emergency keys [16]; Rootstock's PowPeg is an HSM signer federation [17]. Drivechain (BIP-300/301 [18]) is not activated. BitVM-family bridges reduce *safety* to 1-of-N setup honesty plus a challenge game but keep *withdrawal liveness* dependent on at least one operator who fronts funds [7]. Postfork uses that construction. It adds one limit zkBTC does not have: Postfork has its own consensus, so the peg-out verifier must follow both a Bitcoin and a Postfork light client. The peg can be no stronger than the weaker of the Bitcoin-side reserve and what the verifier can establish about Postfork. This paper states that residual plainly.

2. Templates

We define a template as a commitment, made in a coin's script, to the shape of the transaction that spends it. `OP_CHECKTEMPLATEVERIFY` (opcode `0xb3`, formerly `OP_NOP4`) implements it in every script context: bare, P2SH, P2WSH and tapscript. When it executes with a 32-byte top stack element, it computes a single SHA256 over the spending transaction's version, locktime, a hash of scriptSigs (only if any is non-empty), the input count, a hash of all sequences, the output count, a hash of all outputs, and the index of the input being validated. If the result differs from the stack element, the script fails. Any other argument length is a NOP in consensus and non-standard in policy, which keeps those lengths free for future upgrades. The opcode does not pop its argument.

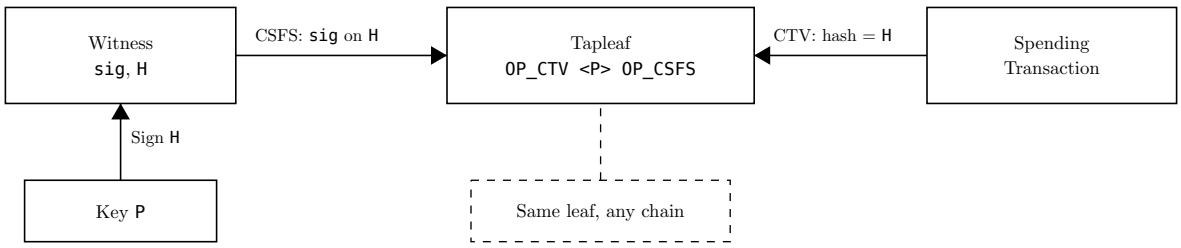
Input outpoints, spent amounts and witnesses are not committed. Templates are the basis of covenant uses such as payment trees, vaults and Ark-style timeout trees. The omission of amounts is also a footgun: an underfunded single-input template output is unspendable forever, and an overfunded one donates the excess to fees [2]. Wallets must check the funding amount against the template and must not reuse a template address for a different amount. Fees must be pre-committed or paid by an anchor output or by committed extra inputs. The bare form `<32-byte hash> OP_CHECKTEMPLATEVERIFY` is a standard output type, as BIP-119 recommends.



3. Signatures from the Stack

`OP_CHECKSIGFROMSTACK` (opcode `0xcc`, formerly `OP_SUCCESS204`) verifies a BIP-340 signature over an arbitrary message taken from the stack rather than over the spending transaction. It exists only in tapscript. It pops a public key, a message and a signature. A 32-byte key with a non-empty signature is checked as BIP-340 [5] over the unhashed message, and a failed check fails the script. An empty signature pushes an empty vector; a key of unknown length succeeds in consensus and is non-standard in policy, as BIP-342 provides for upgrades. Each non-empty signature consumes 50 units of the tapscript validation-weight budget. There is no `VERIFY` variant and no `CHECKSIGADD` equivalent.

Together the two opcodes give rebindable signatures. A leaf `OP_CTV <P> OP_CSFS` accepts a template hash from the witness, checks it against the spending transaction, and checks a signature by key `P` over that same hash. The key holder can sign a new template later, for any coin carrying that leaf. LN-Symmetry-style channels build on this. DLCs with BIP-340 oracle attestations and delegation need only CSFS; DLCs add CTV where a settlement template is fixed. Nothing in the peg constrains which scripts pBTC may use.



Postfork does not include `OP_INTERNALKEY` (BIP-349 [4]); `0xcb` remains `OP_SUCCESS203`. The proposal named "CTV + CSFS" contains exactly BIP-119 and BIP-348 [11], and the Bitcoin Core reference pull requests cover exactly those two [12]. `OP_INTERNALKEY` appears only in larger bundles, LNHANCE and BIP-448 [13]. Including it would track a different future. It adds no spend path that cannot be written for each concrete output, since a script can push the key explicitly; the price is about 8 vbytes per rebindable leaf and the loss of dynamic binding to the output's internal key (one leaf cannot be reused unchanged across outputs with different internal keys, and a rekey does not rebind it automatically). Leaving `0xcb` free keeps the slot open should Bitcoin activate it; adding

it later is a rule change the reserve checks, so it needs a new chain or retired vaults (Section 11). All other proposals, including APO, `OP_CAT`, `OP_PAIRCOMMIT`, `OP_CHECKCONTRACTVERIFY`, `OP_TEMPLATEHASH` and BIP-54, are excluded on the same ground. Postfork implements BIP-119 and BIP-348 as written; the only change is activation, which is always on from genesis.

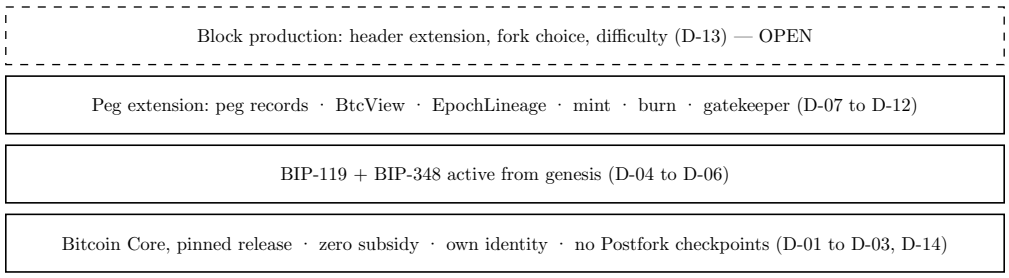
Two hazards follow from the opcodes and are stated rather than hidden. On Bitcoin today, a tapscript containing `0xcc` is an `OP_SUCCESS` script and `<H> OP_CHECKTEMPLATEVERIFY` is `<H> OP_NOP4`: BTC sent on Bitcoin to a Postfork covenant script is lost to the first taker. Wallets must refuse to build such Bitcoin transactions. Second, a CSFS signature over a template hash is valid for any output carrying the same leaf on any chain where the opcodes are active, because the hash commits neither outpoints nor a chain identifier. Wallets must never reuse a CSFS key across chains. Postfork does not add a chain identifier to sighashes or templates, because that would break fidelity.

4. The Chain

Postfork consensus is the consensus of a pinned Bitcoin Core release (provisional base `v32.1` [22]), with every soft fork active on Bitcoin mainnet buried from genesis, BIP-119 and BIP-348 active from block 0, and the peg extension of Sections 5 to 7. Blocks and transactions keep Bitcoin's serialization. Weight, sigops, script limits, signature hashes, Taproot, locktime semantics, coinbase maturity, P2P messages and policy defaults are unchanged; fees and dust are denominated in pBTC satoshis. Proposed Bitcoin soft forks that are not active on Bitcoin, including BIP-54, are not active on Postfork. A rule that is neither Bitcoin Core's nor listed in the delta register is a specification bug.

The block subsidy is zero at every height. There is no halving schedule, no premine, no genesis allocation and no developer fund. The genesis coinbase has only zero-value outputs. Its input script carries the ASCII string `Postfork/v1/genesis` and the peg parameters root, within Bitcoin's 100-byte limit for coinbase input scripts, and a zero-value `OP_RETURN` output carries a pinned Bitcoin checkpoint header. That checkpoint, with its height, chainwork and the ten preceding timestamps, must be a real block of Bitcoin mainnet's most-work chain, namely the most recent retarget boundary at least `D_mint` deep when the genesis is frozen, so that the work floor derived from it is current; every node checks its proof of work at genesis, and its match with mainnet is reproduced independently before launch. The genesis hash is the chain identifier. Before the first mint no pBTC exists and no fees exist; the first mint pays its fee out of the minted value. Supply at any height is the sum of the UTXO set, exactly computable by any full node: minted minus burned minus other provably destroyed value minus fees a coinbase leaves unclaimed.

Postfork must not be confusable with Bitcoin. Its message start bytes, ports (8533 for P2P), address prefix (`pf`) and coin type differ from every Bitcoin network. A Postfork P2TR output and a Bitcoin P2TR output for the same key are byte-identical, so wallets keep the accounts separate. There are no hardcoded Postfork block checkpoints and no checkpoint keys (the genesis-pinned Bitcoin header that roots `BtcView` is a reproducible consensus parameter, not a Postfork checkpoint), no alert key, no emergency key and no admin key. The target block interval should be 600 seconds, so that timelocks in blocks mean what they mean on Bitcoin; difficulty and header extensions depend on block production (Section 9).



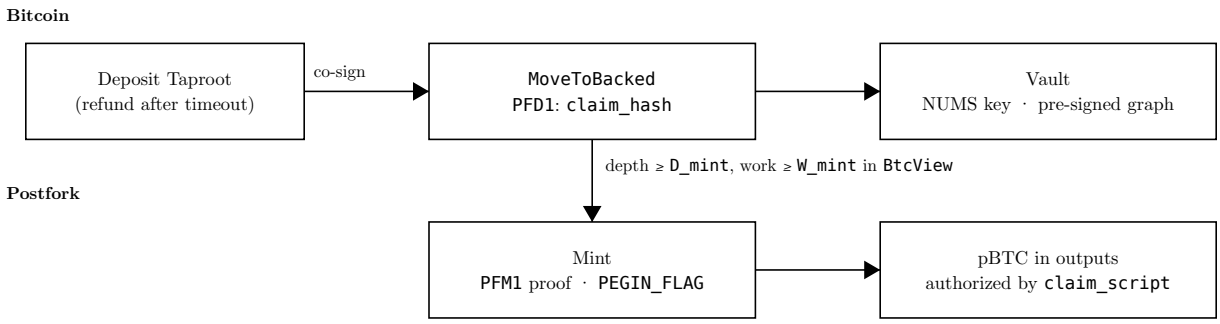
Policy deltas D-06, D-15, D-16 change relay only. Everything not listed is Bitcoin Core.

The delta register has sixteen entries, D-01 to D-16. Conformance requires the BIP-119 and BIP-348 test vectors, the full Bitcoin Core test suite with only the changes the deltas require, and differential tests against Bitcoin Inquisition [12]. Because the Inquisition signet also activates other proposals, among them BIP-54, APO, `OP_CAT`, `OP_INTERNALKEY` and `OP_TEMPLATEHASH`, the comparison is restricted to cases where agreement is expected, and further tests confirm that those proposals stay inactive on Postfork.

5. The Peg

The peg has two halves. The chain defines the statements: a mint is valid if and only if certain facts about Bitcoin hold; a burn requests a payout. The reserve on Bitcoin consumes those statements and never invents its own mint or burn semantics. The peg extension adds no opcode and changes no script semantics. Its rules and the Bitcoin-side reserve do not use CTV or CSFS, with one exception: a mint is authorized by running its claim script under ordinary script rules, so a claim script that uses CTV or CSFS inherits their security in full. Peg data travels in tagged `OP_RETURN` outputs, called peg records: `PFH1` Bitcoin headers, `PFE1` epoch registration, `PFM1` mint proof and `PFB1` burn. Every record has one canonical byte encoding (length-prefixed fields, bounded Merkle branches, no trailing bytes), so every node parses it the same way. Three state objects, `BtcView`, `EpochLineage` and `PegInConsumed`, are part of the chainstate and are undone on reorganizations like the UTXO set.

The reserve is zkBTC's bridge profile. A depositor pays one denomination of the epoch (0.1, 1 or 10 BTC by default), plus exactly the fixed `MoveToBacked` fee `MTB_FEE` and the total value of the fixed fee-anchor outputs (both committed at genesis), into a deposit taproot with exactly two leaves: a leaf spendable only into a pre-signed transaction called `MoveToBacked`, with the depositor's co-signature, and a refund leaf for the depositor after the epoch's refund timelock (200 blocks by default). Both the denomination set and the refund timelock are registered per epoch in a canonical byte encoding, and the mint rule checks both. The operators of the current epoch pre-sign, N-of-N, the deposit's entire BitVM2 graph, including `MoveToBacked`. That transaction moves the deposit into a vault and carries a `PFD1` record committing the chain, the epoch, the vault output and the hash of the Postfork `claim_script`, the script that will authorize the mint and thereby decide which outputs receive the pBTC. The vault is a Taproot output whose internal key is a NUMS point, so there is no key-path spend. Its leaves commit only to the pre-signed claim, challenge, disprove and connector transactions, plus an unspendable leaf binding the vault to Postfork's chain identifier. There is no refund leaf and no live signing key on the vault. After presigning, every honest operator deletes its epoch signing secret. Once one honest operator has deleted, no coalition can sign anything outside the graph.



`BtcView` is Postfork's view of Bitcoin: a tree of Bitcoin headers rooted at the genesis-pinned checkpoint, relayed in `PFH1` records by anyone, including in coinbase transactions before any fee exists. A header is accepted only if it meets Bitcoin mainnet proof of work and the difficulty Bitcoin requires on its own branch, the median-time-past rule, a future-time bound relative to the Postfork block, and Bitcoin's version rules. No header is rejected for being far behind the current best chain, so withheld honest headers can always be relayed later and restore the most-work chain, as long as Bitcoin's chain still contains the pinned checkpoint. A Bitcoin reorganization that discarded the checkpoint would orphan every settled deposit; nodes would then stop treating mints as final, and the chain would have to restart with a new checkpoint. The best chain is the branch with most cumulative work. Wrong headers fail Bitcoin's own rules; forging a chain requires real Bitcoin proof of work. A full node configured with its own Bitcoin node compares the two and warns, and stops reporting new mints as final, if they disagree at depth.

The reserve uses only constructions proven on Bitcoin mainnet. BitVM2-family bridges are live (Bitlayer since 2025-07, Citrea Clementine since 2026-01 [8], [19], [20]), and the Bitcoin-side peg depends on no inactive soft fork, including CTV and CSFS. Unlike some reference deployments, Postfork has no security council.

6. Minting

A mint transaction spends a virtual input. Its first input names the vault outpoint with bit 30 of the output index set (`PEGIN_FLAG`), a technique Elements has used since 2018 [14]. Consensus creates, for the validation of that input only, a virtual prevout whose value is the vault amount and whose script is the committed `claim_script`, which input 0 must satisfy: `claim_script` is the mint authorization script. It binds where the pBTC go only if the authorization commits to all mint outputs (`SIGHASH_ALL`, the default Taproot sighash, or a CTV hash); with `SIGHASH_NONE` or a loose `SIGHASH_SINGLE`, consensus still accepts the mint but anyone who sees it can redirect the uncommitted value. Full binding is a wallet rule, not consensus. The mint is valid only if every clause holds. The `PFM1` record proves that `MoveToBacked` is included in a Bitcoin block on the `BtcView` best chain, with txid and witness Merkle branches and a coinbase proof of equal branch length that rejects the 64-byte-transaction ambiguity. It also carries the deposit transaction, whose txid must match the input `MoveToBacked` spends, so consensus checks that the spent output really is the Taproot key over the deposit and refund leaves; witness data alone would not prove that. Consensus then executes the `MoveToBacked` spend in full against that output, including every signature, each of which must use the default or `SIGHASH_ALL` sighash so that it commits to all outputs, because a header-only view of Bitcoin does not show that Bitcoin validated the transaction. Whether the deposit output really existed unspent stays unproven; a fabricated one would need every operator of the epoch

to sign it, and honest operators check the deposit on their own Bitcoin node before presigning. That block has depth at least `D_mint` = 2016 blocks, and the chainwork from that block (inclusive) to the tip is at least the genesis-pinned `W_mint` (2016 blocks at the checkpoint's difficulty). That normally takes about two weeks, longer if Bitcoin's difficulty has fallen below the checkpoint's. The vault output is byte-equal to the vault template instantiated for Postfork, the epoch, its registered aggregate key and its registered timelock windows, and its amount is one of the epoch's registered denominations; the deposit carried exactly that amount plus `MTB_FEE` plus the anchor outputs' value; any other amount cannot mint. `MoveToBacked` spends the deposit through the pre-signed vault leaf, so BTC paid straight to a vault-shaped script cannot mint, and the deposit's only other leaf is the refund leaf with the epoch's refund timelock. The epoch is registered, mint-eligible and not frozen. Neither the vault outpoint nor the deposit outpoint has minted before; `PegInConsumed` records both, so two different `MoveToBacked` transactions for one deposit cannot both mint. The first input satisfies `claim_script` under ordinary script rules. When a gatekeeper is configured, its signature is present.

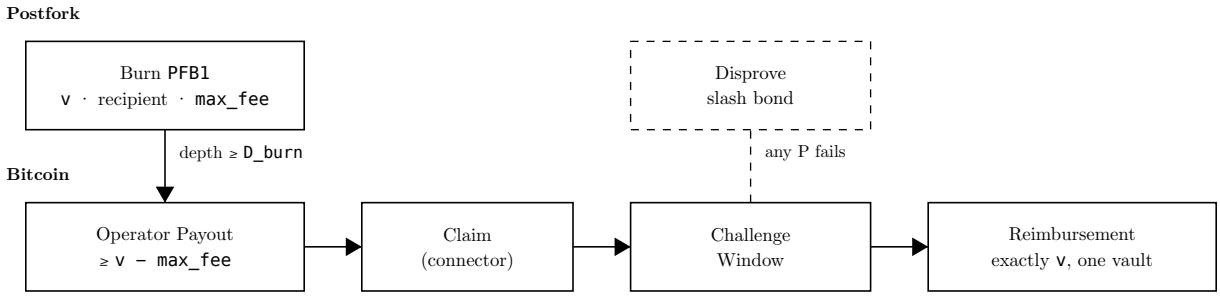
Value created is exactly the vault amount; the difference to the outputs is an ordinary fee. `claim_script` may be any script that can authorize the mint, but not a bare CTV template: the mint transaction carries the proof record as an output, CTV commits every output, and that record contains `claim_script` itself, so the template hash would have to commit to itself. Coins reach a covenant either through a claim leaf that checks a CTV hash signed with CSFS after the proof is known, or by minting to a key and spending into the covenant afterwards. A third party who copies a mint proof gains nothing as long as the holder's authorization commits to every output of the mint transaction: the value can only go where that authorization sends it, and `claim_script` was fixed in the depositor-co-signed `MoveToBacked`. Wallets therefore sign mints with `SIGHASH_ALL` or the default Taproot sighash, or bind the outputs through a CTV hash; a `SIGHASH_NONE` or loose `SIGHASH_SINGLE` mint can be redirected by whoever sees it first. In zkBTC, a mint is bound to its vault by an in-circuit Bitcoin light-client proof, a one-shot mint key and, when designated, a gatekeeper as canonical-chain observer. Postfork replaces all three by consensus: `BtcView` is the canonical observer, `PegInConsumed` is the one-shot, and `claim_script`, the mint authorization script, is the recipient binding. Every such change is recorded as a deviation from zkBTC with its reason. Wallets never build, and honest operators never pre-sign, a deposit into an epoch that cannot back a mint, because once `MoveToBacked` fires such a deposit would be an irrevocable reserve contribution with no pBTC issued.

Attack A, a private Bitcoin fork in which `MoveToBacked` fires only there, now has to beat the most-work rule that every full node applies to the same relayed headers. An attacker must outwork the honest Bitcoin chain over that window (a majority does so reliably; a smaller miner keeps the usual depth-dependent, non-zero chance of catching up), or block producers who withhold honest headers (Attack H) *plus* a private branch of depth `D_mint` carrying at least `W_mint` of chainwork. Because of Bitcoin's timewarp weakness, such a branch can reach that depth with far fewer contemporary block-works than `D_mint` (a forward-dated retarget lowers its difficulty up to fourfold), which is why the work floor, not the depth, bounds the cost; and as Bitcoin's difficulty rises after genesis, `W_mint` corresponds to fewer contemporary blocks. The residual stays Bitcoin-class in cost, but the second path needs no Bitcoin majority, only colluding producers and a miner willing to spend that work; mint canonicity therefore depends on block production too, and a mint settled in such a window is not undone. The work floor bounds the cost per attack branch, not per mint: one private branch can carry many `MoveToBacked` transactions, so the gain can grow with the vault capacity fired on it

while the cost stays the same. The specification accepts that exposure and records a per-window mint cap only as a candidate mitigation. It is not closed further. A Bitcoin reorganization that orphans a settled `MoveToBacked` (`D_mint` or more blocks; exactly `D_mint` suffices against one accepted at depth exactly `D_mint`) leaves minted pBTC unbacked: consensus never re-evaluates a settled mint. That is accepted, as in zkBTC.

7. Redemption

Redemption is burn-first. A holder publishes a `PFB1` record whose value `v` is the redeem amount and whose payload names a Bitcoin key `btc_recipient` and a fee ceiling `max_fee`. The key must be a valid curve point, `v` must be at least `BURN_MIN` (0.1 BTC), and `max_fee` must be below `v`. The burned value leaves circulation, and the burn outpoint is the redeem identifier. There is no unburn. Once the burn is `D_burn` deep on Postfork, any admitted operator with a live bond and a valid sequencing connector for a matching vault's epoch may front the payout from its own funds: a Bitcoin transaction paying `P2TR(btc_recipient)` at least `v − max_fee`, with SegWit inputs solely authorized by that operator, signed `SIGHASH_ALL`, and carrying one claimant commitment to the operator's identity and the burn outpoint. The operator then claims reimbursement along the presigned graph, through its sequencing connector. The claim is valid only for a registered, bonded identity of the vault's epoch, and only once that identity's previous claim has resolved; a successful disprove consumes its bond and connector. A slash spends the identity's connector without a successor, so after a slash no claim of that identity can exist on Bitcoin. The return of an epoch's bond also spends the connector and may re-create it only for the identity's other live epochs, so after a return no claim against that epoch's vaults can exist. Each payout input is checked against its actual prevout, not only its witness. After a challenge window of 1064 Bitcoin blocks without a successful disprove, it receives exactly `v` from one vault. Fees never come from vault value. The gatekeeper appears nowhere in this path.



The BitVM2 path that authorizes a reimbursement must verify a statement whose conjuncts are, in short:

```
P1  the burn exists in Postfork chain C_pf and is a well-formed PFB1 record
P2  C_pf headers are valid under the block-production rule in force
P3  C_pf is canonical against any counter-chain posted in-window that
    itself satisfies P2 and P4 (invalid counter-chains are ignored)
P4  a validity proof shows every block of C_pf valid from genesis
P5  the burn is at depth >= D_burn
P6  max_fee < v; v - max_fee is computed exactly
P7  the payout pays P2TR(btc_recipient) >= v - max_fee on the Bitcoin chain
            with most cumulative work against any counter-chain; every header of the
    claimed and of each counter-chain meets Bitcoin's proof-of-work,
    difficulty, median-time-past and version rules (no local-time check), all chains descend from
the
    genesis checkpoint and count work from it
```

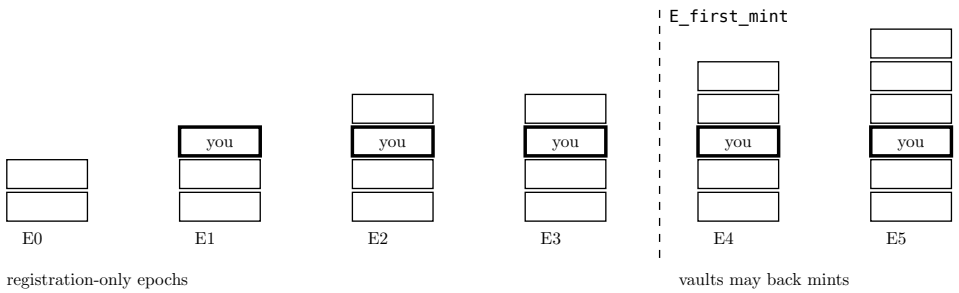
```
P8  one claimant commitment; inputs sole-funded; SIGHASH_ALL; the claimant
    is a registered, bonded identity of the vault's epoch, claiming through
    its own sequencing connector
P9  exactly v leaves exactly one in-set vault
P10 the vault is the template for an epoch registered on C_pf
P11 no earlier valid, unchallenged claim for this burn or this payout; the
    claimant's previous claim is resolved
P12 the Postfork genesis of C_pf matches the vault's binding leaf
```

A claim for which any conjunct is false is disprovable by anyone. P2, P3 and P4 are what zkBTC does not need: they verify Postfork itself. P4 requires a validity proof of a Bitcoin-Core-derived chain verified inside BitVM2, which is not mainnet-proven. Without it, a block-production majority or a consensus bug can create unbacked pBTC and drain the reserve (Attack V). That is the class of the 2026-09-06 Liquid incident, in which a consensus bug let an attacker create roughly 4,000 L-BTC without backing and peg out about 4,000 BTC without any key compromise [16]. A deployment without P4 must not be called pBTC.

Burn-first has a cost. A holder who sets `max_fee` too low, or burns when no operator's economics clear it, or burns an amount with no matching vault, holds a destroyed coin with no payout until someone serves. That is a liveness and ransom risk, not merely a delay. Wallets therefore build a burn only for an amount that matches an unspent vault, with `max_fee` at least the recommended ceiling, after the user confirms that the burn is irreversible. A holder who is an admitted operator can serve their own burn from vaults whose epoch set includes them, while their bond for that epoch is live and their sequencing connector valid; a fraud or grief slash ends that capability even though they stay in S.

8. Operators

The operator set S is one cumulative set for the whole chain. Anyone may register to join in a registration window by posting a bond (2 BTC class) with proofs of key possession. Registering is not being admitted: a joiner enters S only when the current members certify an epoch that includes it, and consensus does not force them to. After admission nobody leaves, and an admitted identity cannot be kicked, though its bond can be slashed. Identity keys are never deleted; for each epoch, every member contributes a fresh epoch signing key, aggregated with MuSig2 [6]. Every new vault is N-of-N over the current full S. Each epoch is anchored on Bitcoin and registered on Postfork, where consensus checks that every identity of the previous epoch is still present. From the second epoch on, a second valid certificate for the same epoch would need the signature of every member of the previous epoch, so if one appears, consensus freezes new mints for that epoch onward: safety over mint liveness. The first epoch has no predecessor to constrain it, so a later registration of it is invalid, a second record for it in the same block has no effect, and the first one wins, which is the genesis-capture risk described below, not a freeze.



For a user who is actually admitted to the operator set of the first mint-eligible epoch, this closes Attack B, in which a Sybil epoch controls all N keys, pre-signs a malicious graph, drains its own vault and redeems unbacked pBTC against honest vaults. Under genesis capture, described below, Attack B stays fully open for anyone whose admission was censored. An honest member never signs a malicious graph, and once one honest identity is in S, no later vault can have one. Postfork adds a consensus rule zkBTC does not have: vaults of epochs below `E_first_mint` (at least four weekly epochs: the schedule starts 144 to 1008 Bitcoin blocks after the genesis is frozen, the chain must launch before that height or be re-pinned, and consecutive epoch anchors must be at least one epoch apart) cannot back a mint. Everyone admitted in that window is in every mint-eligible vault. This does not stop a party that captured the first epoch: admission needs the current members' certificate, and consensus does not force joiners in, so a captor can censor honest joiners and keep every mint-eligible epoch Sybil-only. You must therefore check, on your own node, that you or an operator you trust are actually in the operator set of the first mint-eligible epoch. Coins minted against vaults without an operator you trust remain an untrusted prefix for you, because pBTC is fungible.

Effectively trustless. pBTC is effectively trustless in one precise sense: you do not trust other operators with your bitcoin. You are an operator. You run your own Postfork and Bitcoin full nodes, and your wallet trusts them, exactly as a Bitcoin wallet trusts your own node. Join S before the first mint-eligible epoch and verify that you were admitted, refuse malicious graphs, and exit yourself from vaults whose epoch set includes you, while your bond for that epoch is live and your sequencing connector valid; a fraud or grief slash ends that capability even though you stay in S. Operators then cannot steal under 1-of-N setup honesty, at least one honest live challenger acting within every window, sound circuit, graph and validity-proof cryptography, a Bitcoin adversary below about half the hashrate, and the Postfork-side conditions of the block-production class in force: block producers include honest Bitcoin headers, and the class's canonicity assumption holds (under BP-O1V an honest merge-mining majority); and no Bitcoin reorganization orphans a settled deposit, which would leave minted pBTC unbacked while its burns are reimbursed; and no Postfork reorganization orphans an already reimbursed burn (its pBTC would return while the vault BTC is gone). It does not mean zero assumptions, and it does not mean trusting a company, a federation or a gatekeeper. It does not make block producers irrelevant: producers who keep honest Bitcoin headers out can settle consensus-valid but unbacked pBTC whose burns the reserve would reimburse (Attack H), and being your own operator does not protect against that. It does not make the Postfork chain live, and it does not make a frozen coin recoverable. The claim may not be made for any deployment before every launch gate clears. The cost is real: at product defaults, 2 BTC locked per epoch until no claim path remains, plus fees and ceremony time. The specification's prominent claim, verbatim:

> *pBTC is effectively trustless only in the self-operator sense of §1.1.1, only for the block-production class in force and while its assumptions hold, including that block producers include honest Bitcoin headers, and only after all gates clear. Under the stated assumptions, including that no Bitcoin reorganization orphans a settled deposit, no Postfork reorganization orphans an already reimbursed burn, and no Attack H, F or V succeeds, the worst case is freeze, not theft, and a freeze may be permanent.*

Operators earn redemption fees within `max_fee` and put their bond at risk. A successful disprove pays the challenger `min(bond, max(0.16 BTC, bond/10))` in the slash transaction itself; the rest goes to

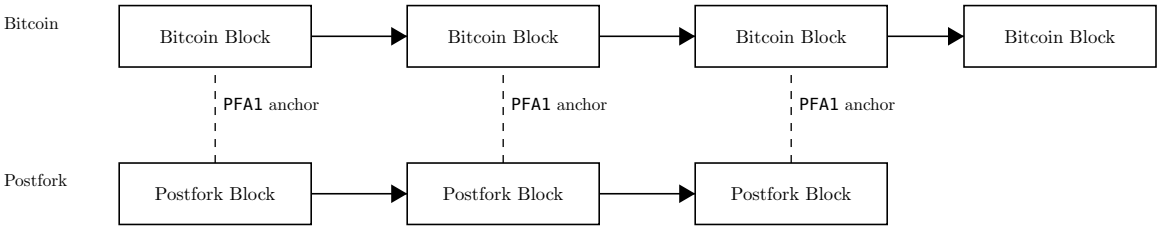
an unspendable output, never to other operators. Sequencing connectors let an operator's next claim proceed only after the previous one resolves. At least three independent watchtowers are a stated floor. Permissionless challenge makes challenging possible; safety requires someone to actually do it.

The gatekeeper is optional and its product default is none. When set, it is pinned in genesis and can only refuse new mints, for example to screen the source of funds. It has no freeze list, no clawback, no role in transfers, burns, payouts or challenges, and no power to redirect a mint, provided the mint is signed with an authorization that commits to every output; a partial-sighash mint can be redirected by anyone who sees it, the gatekeeper included. Because there is one pBTC, it censors all new peg-ins of the chain, and a gatekeeper that refuses after `MoveToBacked` has fired turns that deposit into an irrevocable reserve contribution. It is not what makes pBTC trustless. One configuration is forbidden outright: open registration with a pooled reserve but *without* the growth-only set is unsound and must never be deployed or marketed as trust-minimized.

9. Block Production (open)

How Postfork blocks are produced is an open decision. No option is adopted. Every option must give no party power over value, earn only fees, keep Bitcoin's block format with at most a header extension, keep block validity deterministic, produce blocks before any fees exist, allow header relay in the coinbase, and make P2 and P3 expressible as a BitVM2-verifiable statement. The options are: O1, merge mining (AuxPoW) as specified for Namecoin [15]; O2, an own SHA-256 proof of work; O3, a k-of-n signing quorum using signet's block challenge (BIP-325); H1, O1 plus non-binding pre-confirmations; H2, O3 plus the validity proof; and H3, Bitcoin-anchored fork choice, in which Postfork block hashes are committed on Bitcoin and the canonical chain follows, per height, the first valid anchor in Bitcoin's most-work chain.

Option	**Burn canonicity (P3) holds while ...**	**Without validity proof (P4)**	**Class**	**Verdict for pBTC**
O1	attacker has under half of merge-mining work	merge-miner majority can mint invalid pBTC	weaker	permitted only as BP-O1V (with P4, all gates), with stated residual; without P4 transitional profile, not pBTC
O2	attacker has under half of Postfork-only hashrate	as O1	much weaker	must not ship
O3 / H2	no k-of-n equivocation (Attack Q)	quorum can sign invalid blocks (H2: closed)	quorum	transitional profile, not pBTC; no "effectively trustless" claim
H3	Bitcoin adversary under half the hashrate (inherited)	anchors fix order, not validity	equal to zkBTC	permitted only as BP-H3V (with P4, all gates); without P4 transitional profile, not pBTC



The criteria are reserve-safety class, mainnet-proven components, minimal consensus delta, verifier simplicity, bootstrap safety at zero fees, liveness, and no council or admin key. The non-binding recommendation is H3 for fork choice, so that burn canonicity reduces to Bitcoin as in zkBTC; O1 as the default way blocks are proposed, so that Bitcoin miners can earn pBTC fees; the validity proof as a blocking gate in every case; and rejection of O2 for a real-money peg and of O3 as a safety source. H3 has open problems: data availability for anchored blocks whose data is withheld, who pays the BTC cost of anchors, and a fork-choice rule that is new on a value-bearing chain. If H3 is judged too large a deviation, O1 is the fallback, with the stated residual that burn canonicity rests on the Postfork merge-mining majority. Merge mining does not inherit Bitcoin's security [15]. Mints depend on block production as well: for liveness, and for canonicity through Attack H, because producers who keep honest Bitcoin headers out of their blocks decide what `BtcView` sees. Bitcoin anchoring under H3 does not change that.

10. Security Assumptions

The attacks the specification names, and what closes them, are:

Attack A	mint on a private Bitcoin fork	BtcView most work + D_mint + W_mint (Bitcoin-class)
Attack H	honest headers withheld	any honest relayer's inclusion; node safety mode (rests on block production)
Attack B	Sybil epoch drains its own vault	growth-only set + E_first_mint, only for a user admitted to the set of E_first_mint; open under genesis capture
Attack F	private-fork burn reimbursed on Bitcoin, or a later reorg that orphans a reimbursed burn	P2/P3 + D_burn bound the private fork; late orphaning remains residual 14
Attack V	invalid-state mint, then burned	P4 validity proof (gate G4)
Attack Q	quorum signs two forks	only by not using a quorum (O3/H2)
Attack S	producers censor	none in consensus (liveness)

The specification lists twenty-two residual assumptions. The load-bearing ones are 1-of-N setup honesty; one live admitted operator with a live bond and connector for a matching vault's epoch, for exit; one honest, live challenger acting within every window; a Bitcoin adversary below about half the hashrate, Bitcoin's own base assumption, inherited; deep finality at `D_mint` with the work floor `W_mint`; one honest header relayer whose headers block producers include; sound cryptography and validity proof; burn canonicity at the strength of the block-production class; and a fee-only security budget. The premise itself is a risk: Bitcoin may activate a different covenant set, different revisions, or nothing, and then Postfork contracts do not port one-to-one. The specification states when pBTC is safe in one sentence:

> *pBTC is safe for you if you (or an operator you trust) were admitted to the operator set of the first mint-eligible epoch, as verified on your own node, and refuse malicious graphs, at least one honest challenger watches both chains and acts in time, the cryptography and the Postfork validity proof are sound, Bitcoin's hashrate majority is honest and no Bitcoin reorganization orphans a settled deposit, Postfork block producers include honest Bitcoin headers, and the block-production class in force is one that §11.2 permits for pBTC and its canonicity assumption holds (under BP-O1V: an honest merge-mining majority) and no Postfork reorganization orphans an already reimbursed burn — and even then, the worst case is a freeze, which is not theft but may never be recoverable.*

Freeze is not theft; "not theft" is not "recoverable". There is no council, no admin key, no in-place upgrade of a live vault graph and no rescue path. A non-soundness bug can freeze vaults irreversibly. A soundness bug in the graph, the reimbursement statement or the validity proof can enable theft, and so can a consensus bug shared by the nodes and the validity-proof specification. This design refuses the trust root of a security council deliberately and pays for it with irreversibility. Supply is exact; backing is conditional: circulating pBTC stays at or below the unspent vault set only while the assumptions above hold.

We do not repeat the original paper's catch-up calculation [1]. At `D_mint` and `W_mint` the residual that matters is not raw catch-up but which branch `BtcView` and block production treat as canonical, and the specification answers that by attack class, not by a probability.

Postfork adds no privacy and claims none. Deposits, mints, transfers, burns, payouts and operator registrations are public, and the peg links the Bitcoin deposit to the Postfork recipient and the burn to the payout. Users should use fresh keys per deposit, claim script and recipient.

11. Convergence with Bitcoin

Postfork does not track Bitcoin automatically. Every change is a new specification edition, with adversarial review when it touches script, the peg or the trust model. No edition may change the meaning of `0xb3`, `0xcc` or any opcode already defined; existing outputs keep the rules they were created under. If Bitcoin activates BIP-119 and BIP-348 with identical semantics, Postfork has converged and the cross-chain replay hazard becomes live. If Bitcoin activates different revisions, Postfork keeps the pinned ones. If it activates a different bundle such as BIP-448, Postfork may add those opcodes in their Bitcoin slots, which Section 3 kept free, but only as a new chain or once existing vaults no longer back circulating pBTC, and it never removes CTV; it is then a superset and no longer "Bitcoin as it will look." If Bitcoin activates CTV only, CSFS remains a Postfork extension. If Bitcoin activates nothing, the premise stays hypothetical and is described as such. Any change to the rules that the reserve's verifier checks, even a tightening soft fork, is treated like a hard-fork edition that users choose by running it; no key, foundation or vote can impose one. Existing vaults check burns only against the rules their graphs were signed for, and their verifier also accepts blocks that a tightened rule set would reject, so such an edition must be a new chain, reached by burning on the old chain and re-pegging, or must not be adopted while existing vaults back circulating pBTC.

If CTV and CSFS activate on Bitcoin, they can at most replace parts of the pre-signed graph with templates, which may reduce the setup-honesty residual. They cannot verify a Postfork burn or validity proof in script, so the operator-liveness residual stays. With CSFS a committed key can authorize any concrete template, including different amounts, but neither opcode does amount arithmetic or introspection, so a permissionless, proof-derived payout of variable amount still needs an authorizing key or an additional verifier. An exit that anyone can satisfy with a valid Postfork burn proof would need more capability, such as `OP_CAT`-class introspection or BitVM-style verification. Any new vault generation must be shown under adversarial review to be at least as strong as the existing one, because both would back the same fungible pBTC. Cheaper verifiers such as BitVM3, Mosaic and Argo [23] could lower dispute cost and allow smaller denominations once mainnet-proven. None of these is a launch dependency. Lowering `D_mint`, `W_mint` or `D_burn` is a trade-off of safety for latency, never an upgrade.

Two gate families must both clear before anything may be called pBTC. The four requirement gates are: PF-R-09 and anti-domination encoded, with the registration-only epochs enforced in consensus; setup integrity, including an MPC ceremony for any circuit-specific setup; holder-to-operator onboarding implemented in at least one wallet; and challenger economics re-checked against the worst-case dispute cost. All four are open. The maturity gates are: G1, BitVM2 mainnet-proven, cleared; G2, an open operator market for pBTC operational, with verified admission of an unaffiliated operator into the first mint-eligible epoch and a test mint from it; G3, the reimbursement statement for Postfork burns demonstrated, including the binding of each claim to a bonded operator of the vault's epoch and its sequencing connector; G4, the Postfork validity proof demonstrated inside BitVM2; G5, the block-production decision closed; G6, consensus conformance passed and launch pins frozen, with independently reproduced genesis-checkpoint values and test vectors for the epoch-parameter encoding, the mint's denomination and refund-leaf checks and the first-epoch registration rule. G2 to G6 are open. Specifying a rule, an encoding or a number does not clear any gate.

12. Conclusion

We have proposed a chain on which the covenant script layer proposed for Bitcoin is designed to be used with pegged bitcoin and nothing else, once all launch gates clear. Its rules are Bitcoin Core's, with BIP-119 and BIP-348 active from genesis and every other difference listed. There is no subsidy, no premine and no token. pBTC (Postfork) is minted only against a vault deposit that consensus verifies at deep finality on the most-work Bitcoin header chain, and is redeemed by burning first. The reserve is zkBTC's: NUMS vaults spendable only along pre-signed BitVM2 graphs, an open, growth-only operator set, no council and no admin keys. A holder who is admitted before the first mint-eligible epoch is the honest signer of every vault that can back a mint and can serve their own exit while their bond and connector for that epoch are live; admission can be censored, so it must be verified. Under the enumerated assumptions, including that block producers include honest Bitcoin headers, that no Bitcoin reorganization orphans a settled deposit and that no Postfork reorganization orphans an already reimbursed burn, operators cannot steal, and the worst case is a freeze that may be permanent. The peg is only as strong as what its verifier can establish about Postfork: the validity proof is a blocking gate, and block production is an open decision. This paper is an informative introduction; the design is specified normatively in [24] and awaits implementation.

References

[1] "Bitcoin: A Peer-to-Peer Electronic Cash System," https://bitcoin.org/bitcoin.pdf, 2008.
[2] BIP-119, "CHECKTEMPLATEVERIFY," https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki ; Bitcoin Optech Newsletter 2026-01-02 (amount footgun), https://bitcoinops.org/en/newsletters/2026/01/02/.
[3] BIP-348, "CHECKSIGFROMSTACK," https://github.com/bitcoin/bips/blob/master/bip-0348.md.
[4] BIP-349, "OP_INTERNALKEY" (excluded), https://github.com/bitcoin/bips/blob/master/bip-0349.md.
[5] BIP-340, BIP-341, BIP-342 (Schnorr, Taproot, Tapscript), https://github.com/bitcoin/bips.
[6] BIP-327, "MuSig2 for BIP340-compatible Multi-Signatures," https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki.
[7] "BitVM2: Bridging Bitcoin to Second Layers," https://bitvm.org/bitvm_bridge.pdf ; https://bitvm.org/bitvm2.
[8] "Clementine: A Collateral-Efficient, Trust-Minimized, and Scalable Bitcoin Bridge," Cryptology ePrint Archive 2025/776, https://eprint.iacr.org/2025/776.
[9] zkBTC specification `spec/ZKBTC_TOKEN.md`, commit `0f0582a`, https://github.com/zk-BTC/zkbtc ; whitepaper, https://zkbtc.com/white-paper/.
[10] zkCoins documentation, https://github.com/zk-coins/docs ; https://docs.zkcoins.com.

[11] CTV + CSFS open letter, https://ctv-csfs.com/ ; Bitcoin Optech Newsletter 2025-07-04, https://bitcoinops.org/en/newsletters/2025/07/04/ ; Delving Bitcoin, “CTV+CSFS: Can we reach consensus on a first step towards covenants?”, https://delvingbitcoin.org/t/ctv-csfs-can-we-reach-consensus-on-a-first-step-towards-covenants/1509.
[12] Bitcoin Core PR #31989 (BIP-119), https://github.com/bitcoin/bitcoin/pull/31989 ; PR #32247 (BIP-348), https://github.com/bitcoin/bitcoin/pull/32247 ; Bitcoin Inquisition releases, https://github.com/bitcoin-inquisition/bitcoin/releases ; release v29.4-inq, https://github.com/bitcoin-inquisition/bitcoin/releases/tag/v29.4-inq.
[13] Bitcoin Optech Newsletter 2025-12-05 (LNHANCE), https://bitcoinops.org/en/newsletters/2025/12/05/ ; BIP-448, https://github.com/bitcoin/bips/blob/master/bip-0448.md.
[14] Elements peg-in outpoint flag, https://github.com/ElementsProject/elements/blob/master/src/primitives/transaction.h.
[15] Merged mining specification, https://en.bitcoin.it/wiki/Merged_mining_specification ; “Merged Mining: Curse or Cure?”, Cryptology ePrint Archive 2017/791, https://eprint.iacr.org/2017/791 ; BIP-325 (Signet), https://github.com/bitcoin/bips/blob/master/bip-0325.mediawiki.
[16] Liquid technical overview, https://docs.liquid.net/docs/technical-overview ; Liquid Network security incident assessment, https://blog.blockstream.com/liquid-network-security-incident-assessment/.
[17] Rootstock PowPeg, https://dev.rootstock.io/concepts/powpeg/.
[18] BIP-300 and BIP-301, https://github.com/bitcoin/bips/blob/master/bip-0300.mediawiki ; https://github.com/bitcoin/bips/blob/master/bip-0301.mediawiki.
[19] Citrea documentation, mainnet status, https://docs.citrea.xyz/essentials/introduction.md.
[20] Blockworks, BitVM implementation stage, https://blockworks.com/news/bitvm-implementation-stage.
[21] pNetwork post-mortem, pBTC-on-BSC exploit, https://medium.com/pnetwork/pnetwork-post-mortem-pbtc-on-bsc-exploit-170890c58d5f ; The Block, 2021-09-20, https://www.theblock.co/news/defi/2021-09-20-defi-pnetwork-hack-wrapped-bitcoin-12-million-lost-118060.
[22] Bitcoin Core releases, https://github.com/bitcoin/bitcoin/releases.
[23] BitVM3, https://bitvm.org/bitvm3.pdf ; Mosaic, https://eprint.iacr.org/2026/812 ; Argo, https://eprint.iacr.org/2026/049.
[24] Postfork design specification `spec/POSTFORK.md`, edition spec-v1.0 (draft), 2026-11-08; the single normative document. Planned publication under the GitHub organisation “postfork”.